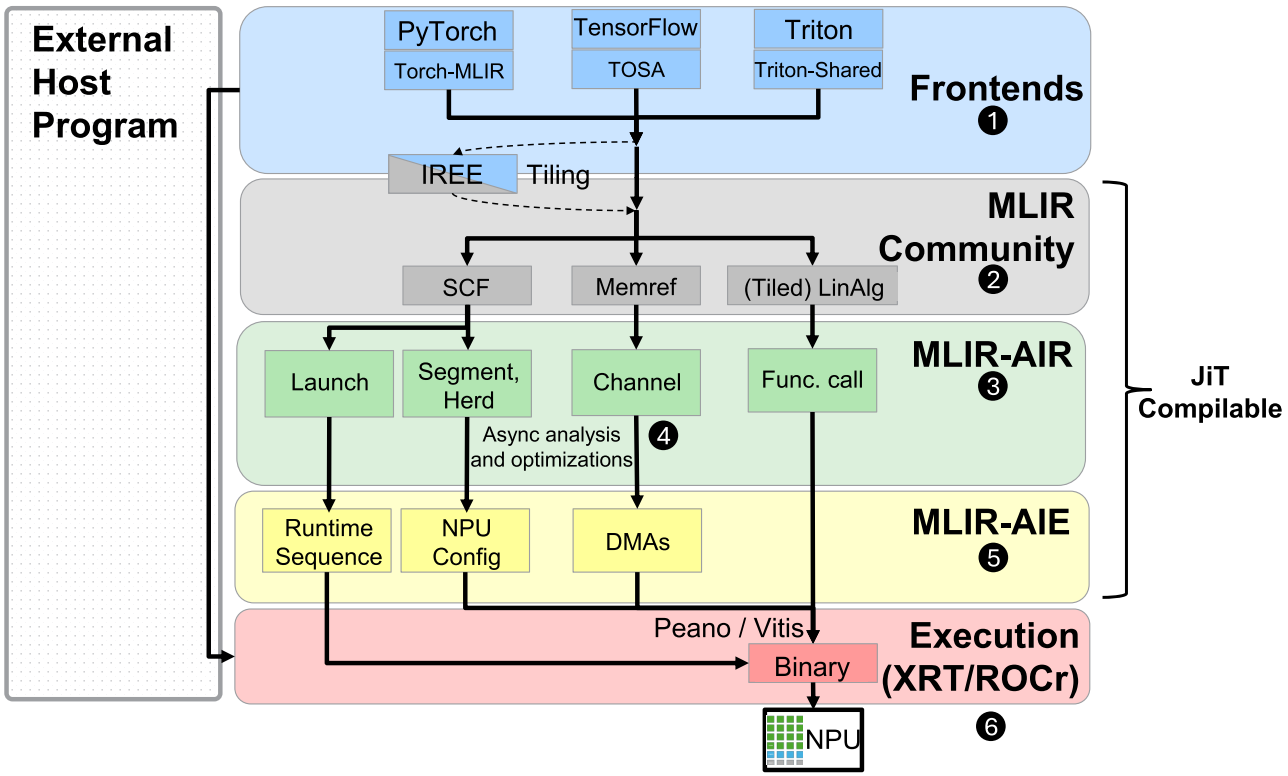
Practical Correctness and Equivalence Checking for MLIR

Emily Tucker[1], Louis-Noël Pouchet[2], Erika Hunhoff[3], Stephen Neuendorffer[3], Erwei Wang[3]

[1]Georgia Institute of Technology [2]Colorado State University [3]AMD

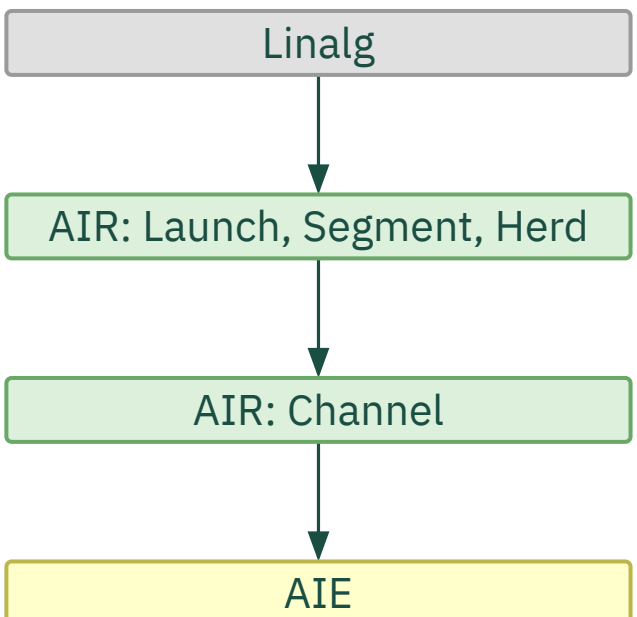
PACT 2026 · October 21, 2026 · Chicago, IL

MLIR-AIR/AIE: Accelerator Design with MLIR



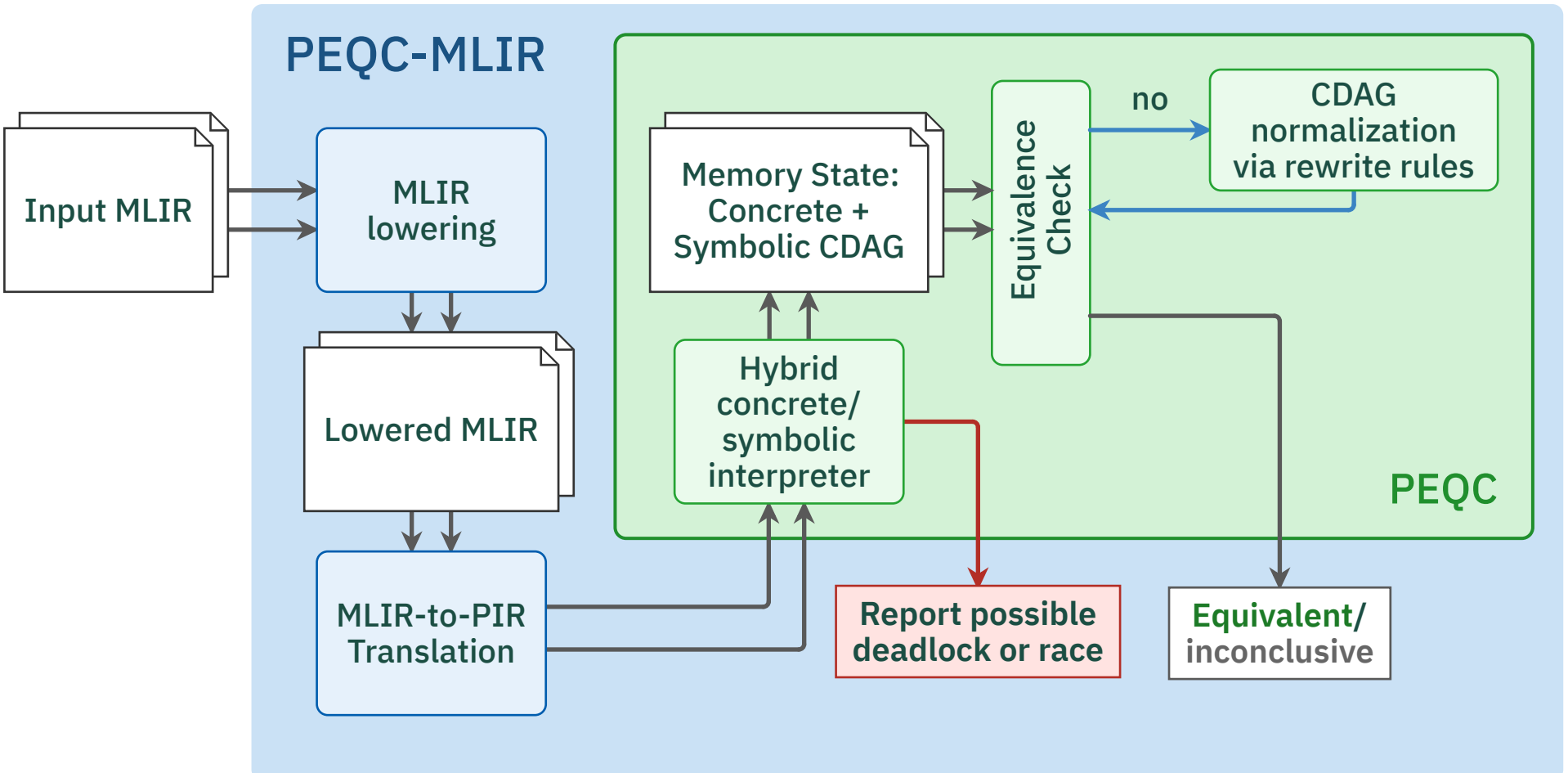
Wang et al. From Loop Nests to Silicon: Mapping AI Workloads onto AMD NPUs with MLIR-AIR. ACM TRETS'26

MLIR-AIR/AIE Compilation Pipeline

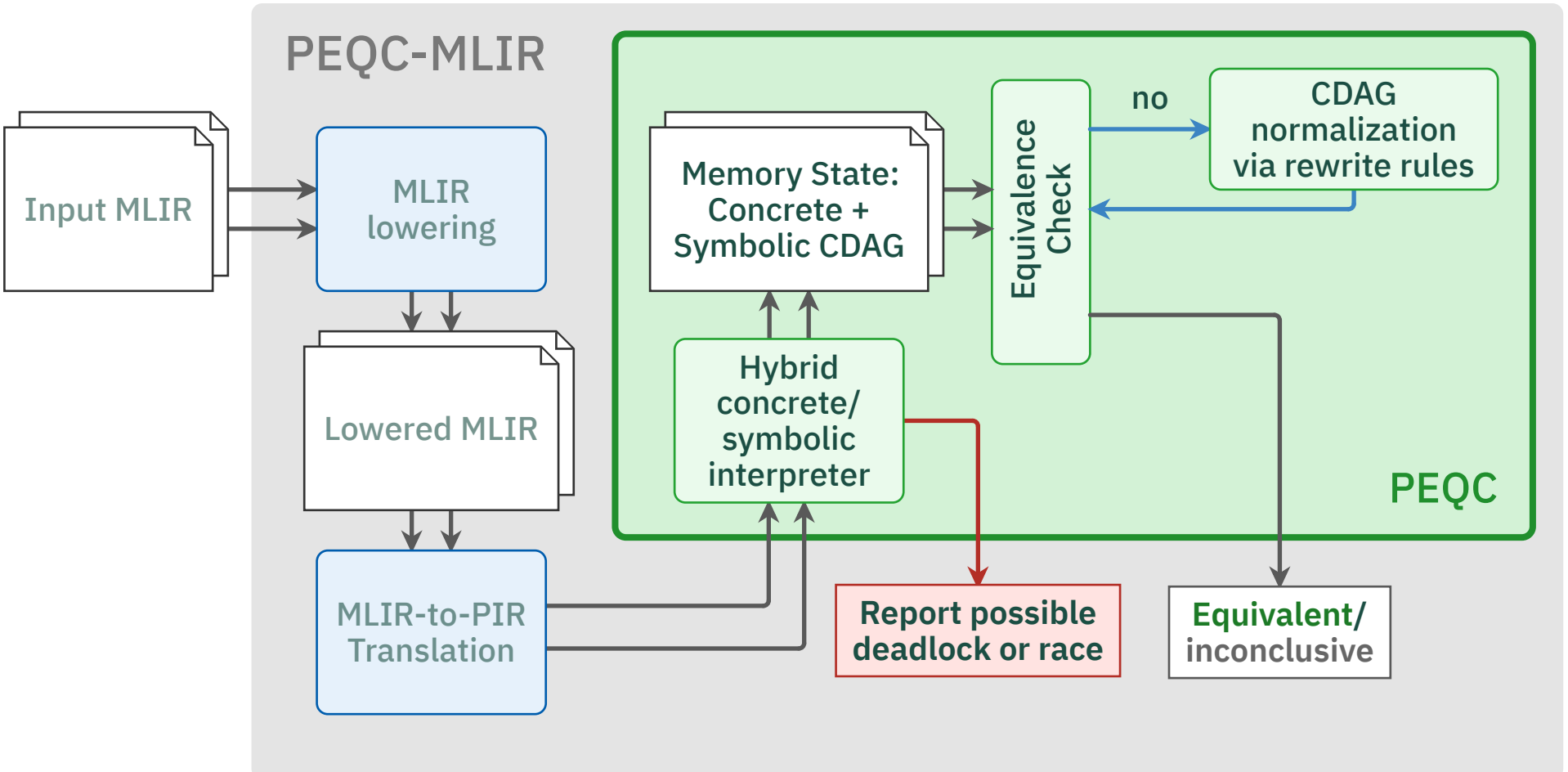


- **`linalg`**: `linalg.generic` matmul
  possible bug: missing/incorrect operation
- **`air.tiled`**: parallelization, 3D tiling
  possible bugs: race condition, incorrect tiling
- **`air.placed`**: add communication with channels
  possible bug: incorrect data layout
- **`aie`**: code per PE, DMA, `objectfifo`
  possible bugs: nondeterminism, deadlock

The PEQC-MLIR Approach: Overview



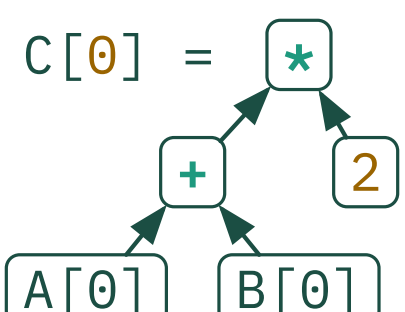
The PEQC Approach: Overview



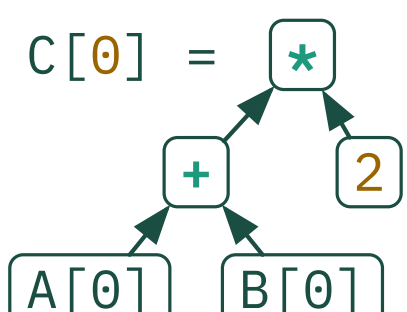
Computation Equivalence with CDAGs

```
for (int i = 0; i < 1 + 1; i++)
  C[i] = (A[i] + B[i])
         * (i + 2);
```

```
float tmp[2]; sem s[2];
async { for (int i = 0; i < 2; i++) {
          tmp[i] = A[i] + B[i];
          set(s[i], 1); } }
async { for (int i = 0; i < 2; i++) {
          wait(s[i], 1);
          C[i] = tmp[i] * (i + 2); } }
```



=



Example: Concurrent Producer/Consumer

```
float tmp[2];

async {
  for (int i = 0; i < 2; i++)
    tmp[i] = A[i] + B[i];
}

async {
  for (int i = 0; i < 2; i++)
    C[i] = tmp[i] * (i + 2);
}
```

Example: Concurrent Producer/Consumer

```
float tmp[2];
async { for (int i = 0; i < 2; i++)
          tmp[i] = A[i] + B[i]; }
```

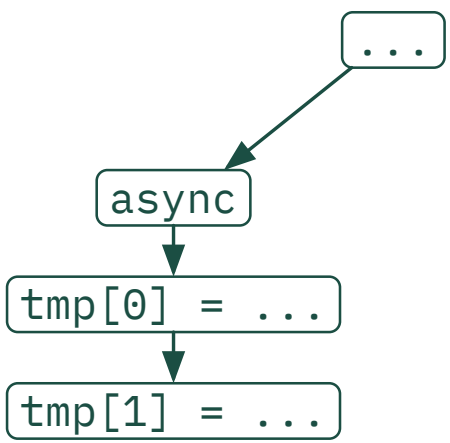
```
async { for (int i = 0; i < 2; i++)
            C[i] = tmp[i] * (i + 2); }
```

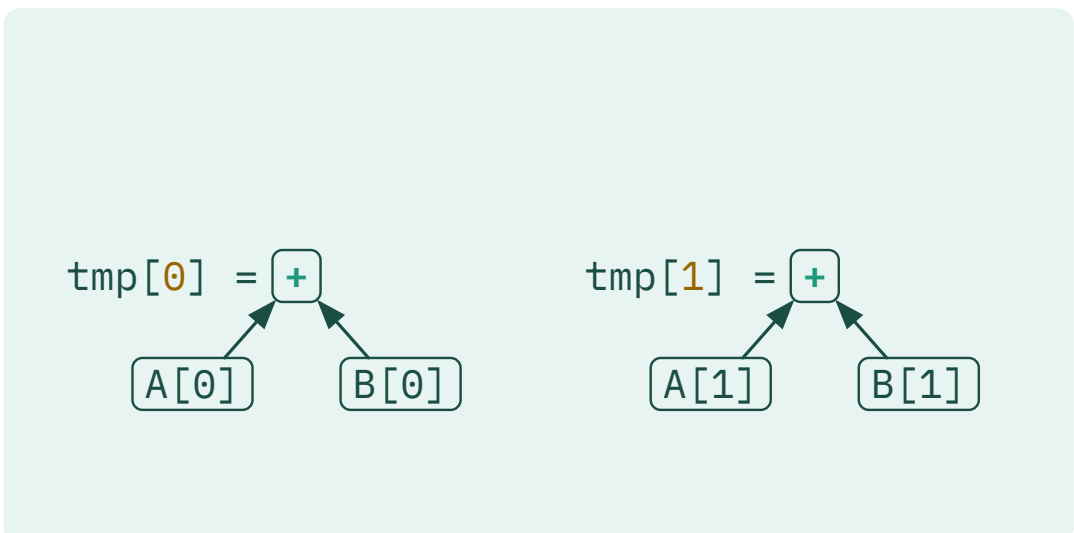


Example: Concurrent Producer/Consumer

```
float tmp[2];
async { for (int i = 0; i < 2; i++)
          tmp[i] = A[i] + B[i]; }
```

```
async { for (int i = 0; i < 2; i++)
            C[i] = tmp[i] * (i + 2); }
```

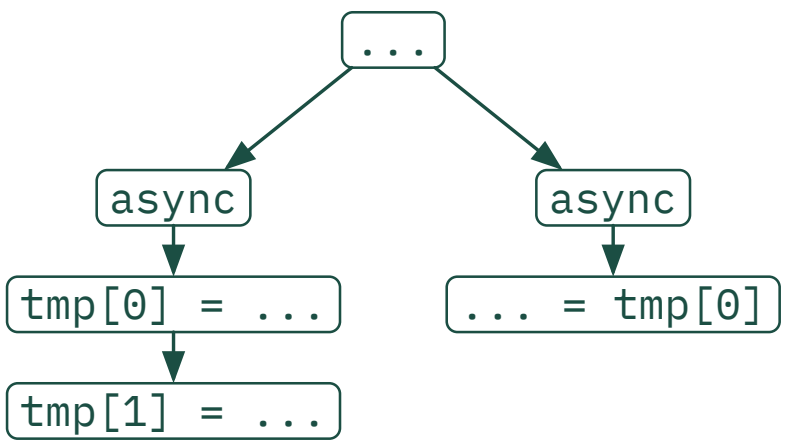




Example: Concurrent Producer/Consumer

```
float tmp[2];
async { for (int i = 0; i < 2; i++)
          tmp[i] = A[i] + B[i]; }
```

```
async { for (int i = 0; i < 2; i++)
            C[i] = tmp[i] * (i + 2); }
```

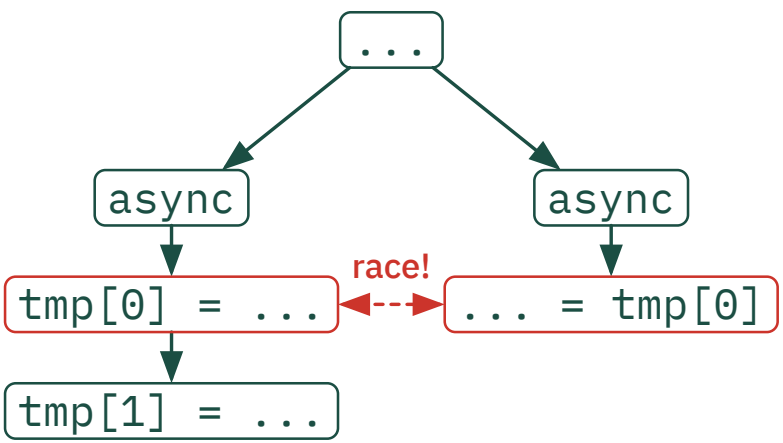


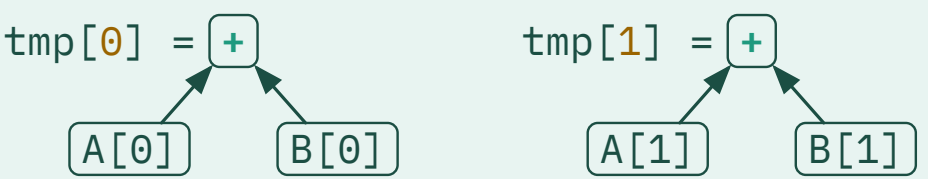


Example: Concurrent Producer/Consumer

```
float tmp[2];
async { for (int i = 0; i < 2; i++)
          tmp[i] = A[i] + B[i]; }
```

```
async { for (int i = 0; i < 2; i++)
            C[i] = tmp[i] * (i + 2); }
```

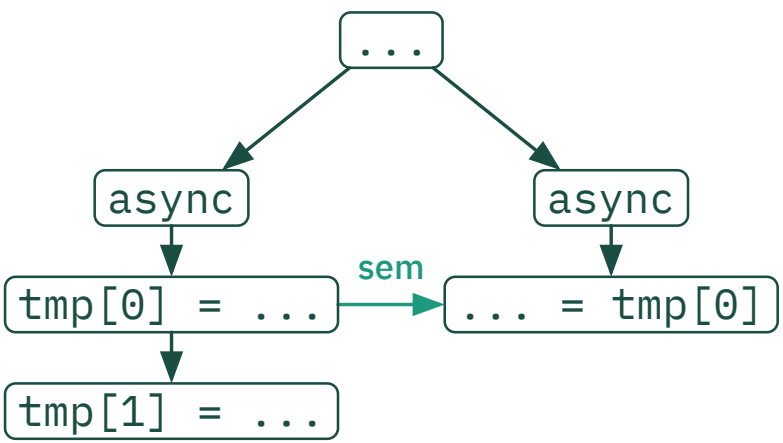


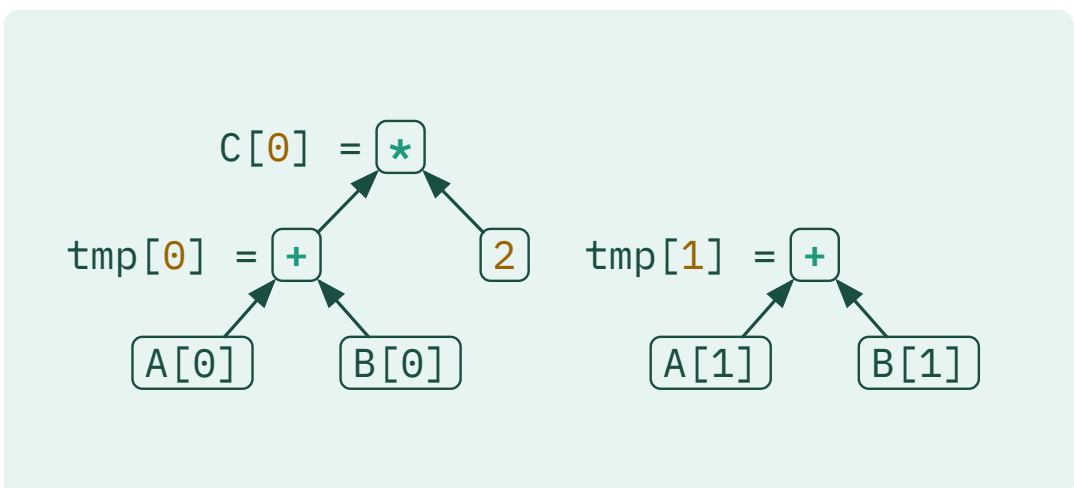


Example: Concurrent Producer/Consumer

```
float tmp[2]; sem s[2];
async { for (int i = 0; i < 2; i++) {
          tmp[i] = A[i] + B[i];
          set(s[i], 1); } }
```

```
async { for (int i = 0; i < 2; i++) {
          wait(s[i], 1);
          C[i] = tmp[i] * (i + 2); } }
```





Restricting Concurrent Programs: Determinism

How to interpret only **one** schedule, but prove equivalence over **all** schedules?

Semaphore restrictions to ensure α-determinism:

- Binary semaphores: one `set` matches one `wait`
- Counting semaphores: no two `acquires` in parallel, quantities match exactly

Data structure: **happens-before graph** built during interpretation

- nodes are statement instances + read/write accesses
- edges describe ordering, edge between statements iff they happen in sequence

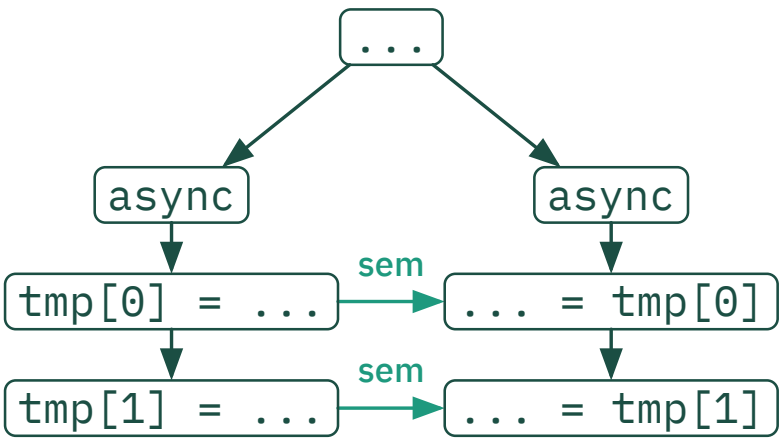
The happens-before graph lets us enforce restrictions and catch races/deadlock

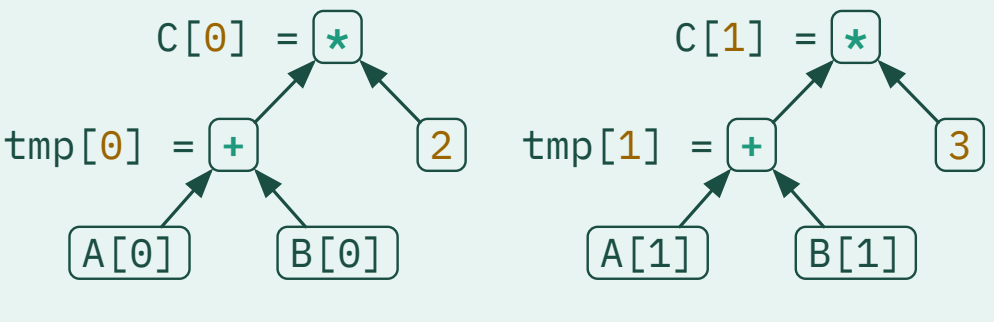
Tucker et al. Practical Formal Verification for MLIR Programs. 2026. arXiv. 32 pages

Example: Concurrent Producer/Consumer

```
float tmp[2]; sem s[2];
async { for (int i = 0; i < 2; i++) {
          tmp[i] = A[i] + B[i];
          set(s[i], 1); } }
```

```
async { for (int i = 0; i < 2; i++) {
          wait(s[i], 1);
          C[i] = tmp[i] * (i + 2); } }
```





Computation Equivalence with CDAGs

```
for (int i = 0; i < 2; i++)
  C[i] = (A[i] + B[i])
         * (i + 2);
```

```
float tmp[2]; sem s[2];
async { for (int i = 0; i < 2; i++) {
          tmp[i] = A[i] + B[i];
          set(s[i], 1); } }
async { for (int i = 0; i < 2; i++) {
          wait(s[i], 1);
          C[i] = tmp[i] * (i + 2); } }
```



Correct Programs

A correct transformed program is **equivalent** to the original program

Properties of equivalent programs:

- **Equivalent CDAGs:** every live-out value is computed by the same operations for **any input values**
  - Different schedules of independent operations are allowed: tiling, parallelization, etc.
- **No concurrency errors:** no race conditions or deadlock for **any possible schedule** that implements synchronizations
  - **race:** two accesses to the same variable ($\geq$1 write), unordered by synchronization
  - **deadlock:** tasks are waiting on synchronization and none can make progress

Pouchet et al. Formal Verification of Source-to-Source Transformations for HLS. ACM FPGA'24

Our Concurrency Model

Our model: asynchronous tasks + synchronization via semaphores

Different types of semaphores to model different behavior:

- Binary semaphores (`set`/`wait`)
- Counting semaphores (`release`/`acquire`)

Zero-latency model: infinite resources, operations take 0 cycles, only synchronization creates order

No race or deadlock with zero latency ⇒ no race or deadlock under **any** latency

General enough to express other models: parallel loops, hardware concurrency

PEQC Takeaways

Scales to practical programs

- Interpretation scales linearly with number of operations **executed**
- Concurrency checking is worst-case quadratic, but average linear

Hybrid concrete/symbolic interpretation

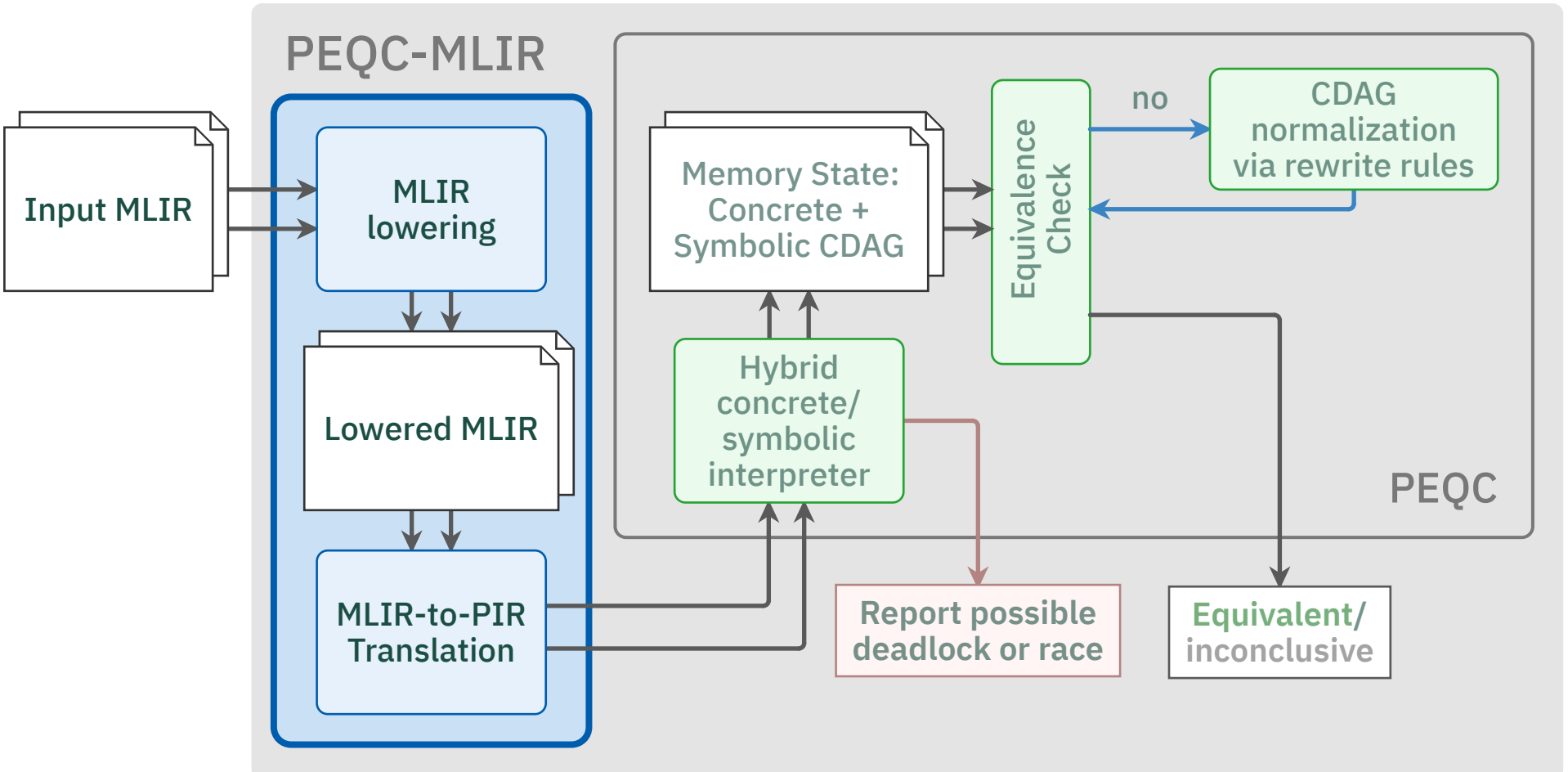
- Syntax independent: not dependent on syntax of programs to build CDAGs
- Uninterpreted math functions (`exp`, `tanh`, ...) for symbolic data / ML workloads

Limitations

- Concurrency must be α-deterministic: no nondeterminism or atomic updates
- CDAG size in memory: limited to e.g. 512×512 matrix multiplication is 100GB

All of PEQC is in the trusted codebase!

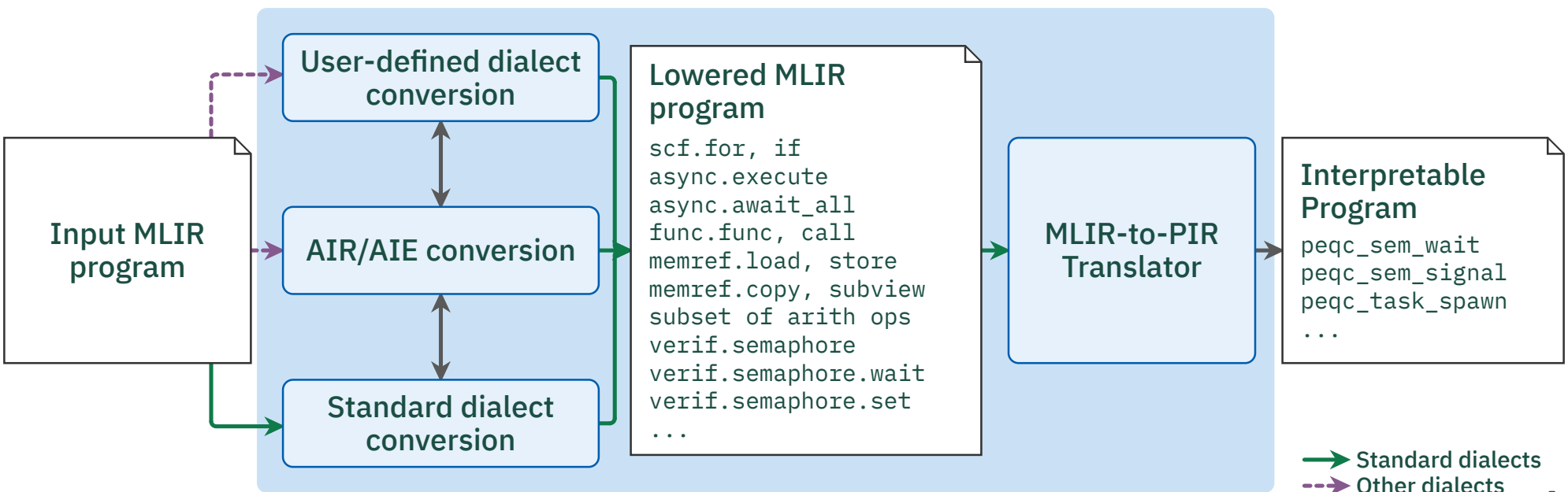
The PEQC-MLIR Approach



The PEQC-MLIR Approach

Semantics and verification via translation:

- Convert all operations to standard dialects + our semaphore operations
- Translate all standard operations to PIR



Defining Semantics for MLIR Operations

We define semantics of MLIR operations through **translation**:
a per-operation conversion algorithm from source dialect to standard subset

Our translation target: standard operations one-to-one with PIR

- general control flow (`cf`, `scf`)
- memory operations (`memref`), arithmetic (`arith`, `math`)
- concurrency (`async`) and our new dialect for semaphores

Contribution: we define operation semantics for AMD's AIR/AIE dialects

AIE Semantics and Conversion

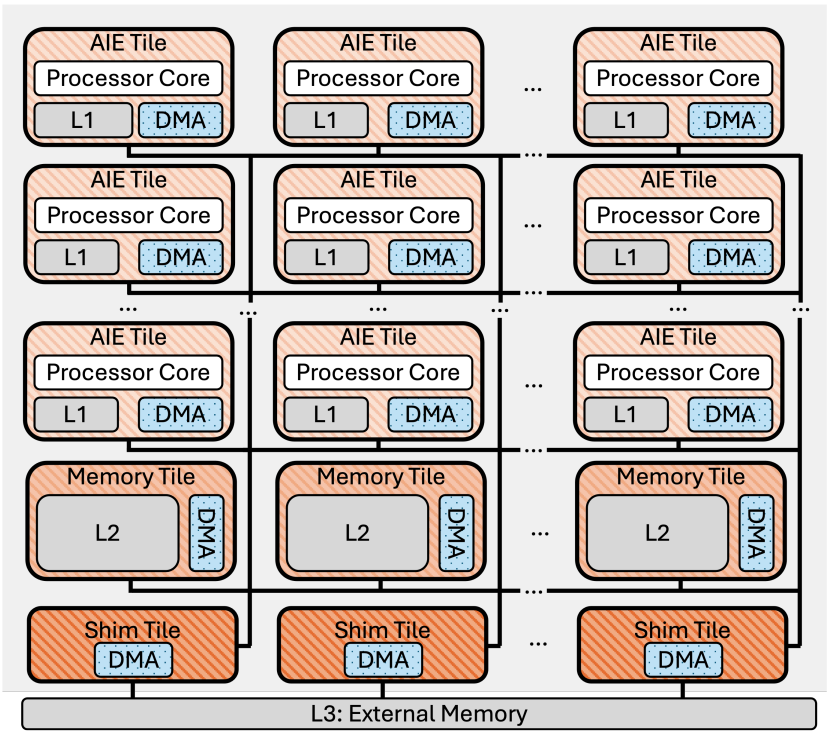
AIE program model

- `aie.core`/`aie.lock`: PEs and fine-grained synchronization

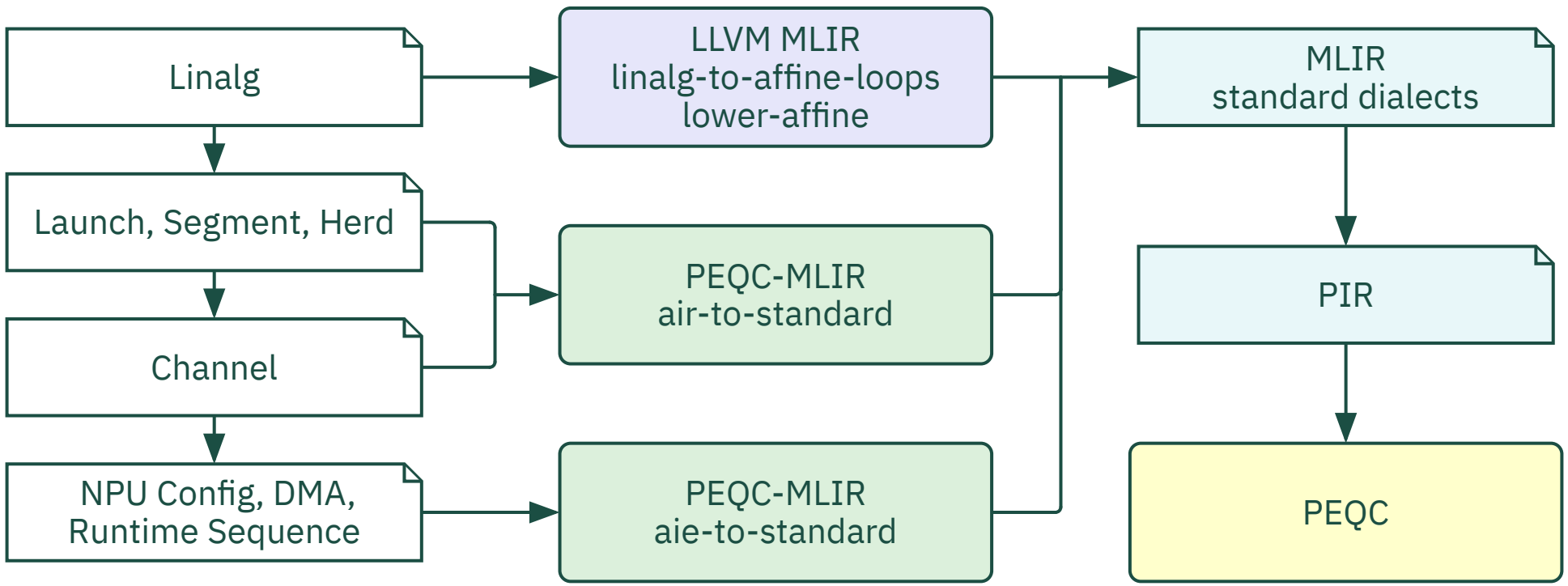
Conversion to standard dialects

- Cores and memory tiles become `async` tasks
- Locks become semaphores
- FIFOs become buffers + semaphores

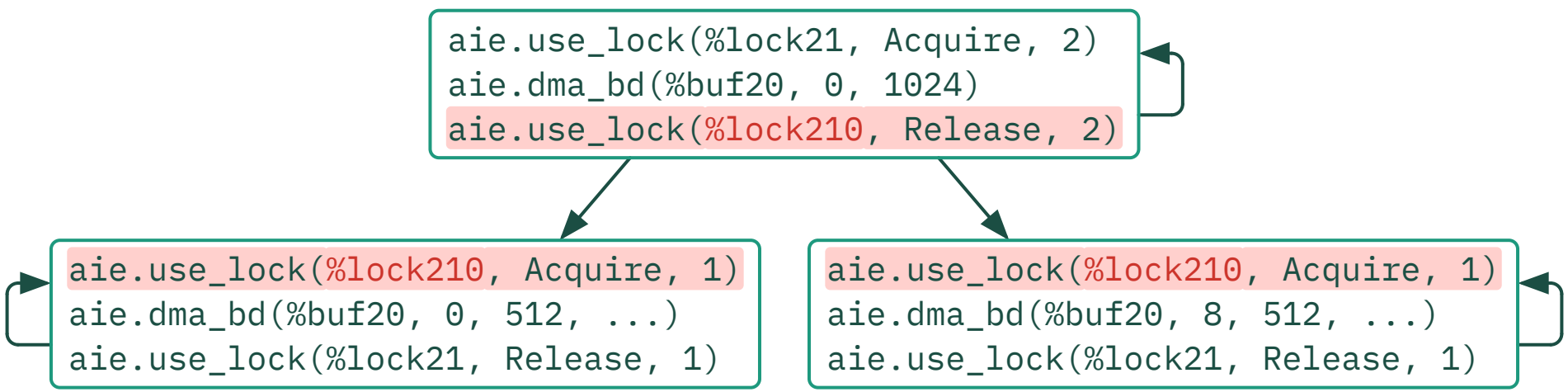
Some overhead of conversion/translation to PIR, number of operations increases



AIR/AIE Verification

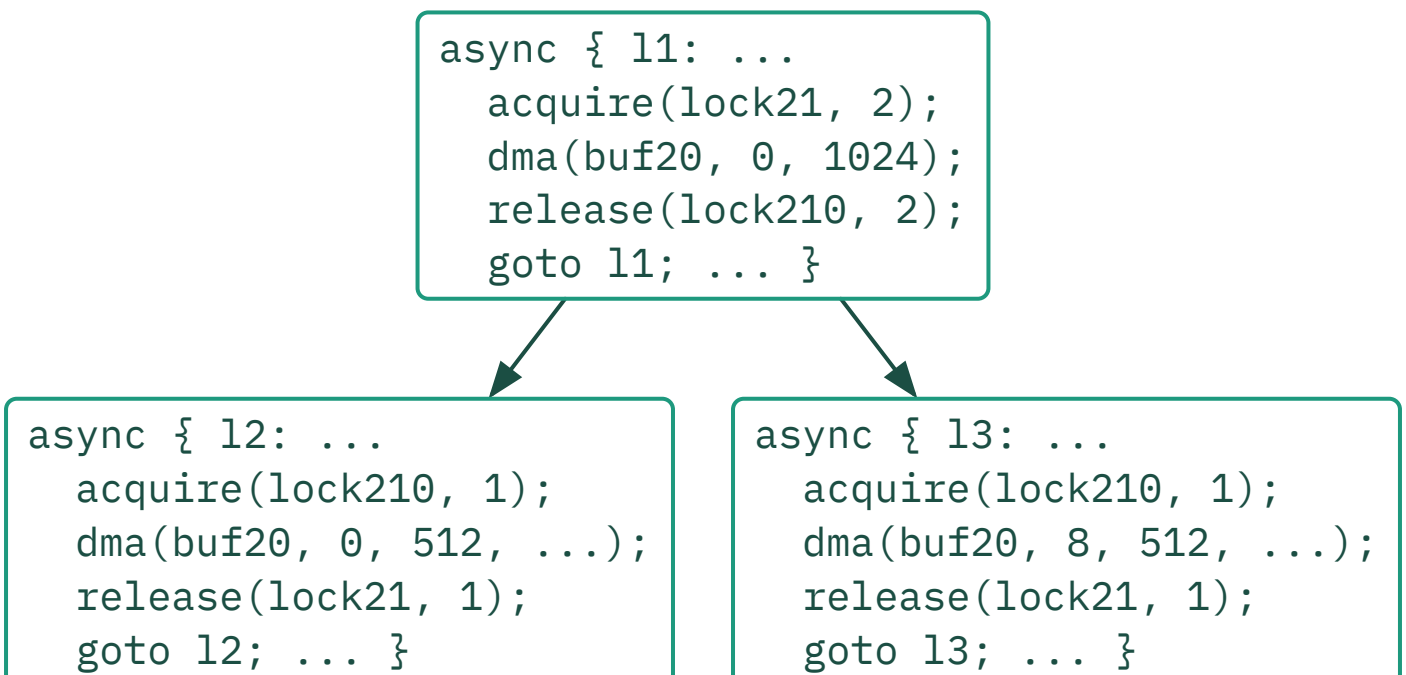


Bug Example: Excerpt from AIE matmul



Bug Example: AIE

Translated from MLIR to PIR: convert `aie.core` to `async` tasks, convert lock operations to semaphore operations



Bug Example: AIE

```
async { l1: ...
  acquire(lock21, 2);
  dma(buf20, 0, 1024);
  release(lock210, 2);
  goto l1; ... }
```

```
async { l2: ...
  acquire(lock210, 1);
  dma(buf20, 0, 512, ...);
  release(lock21, 1);
  goto l2; ... }
```

```
async { l3: ...
  acquire(lock210, 1);
  dma(buf20, 8, 512, ...);
  release(lock21, 1);
  goto l3; ... }
```

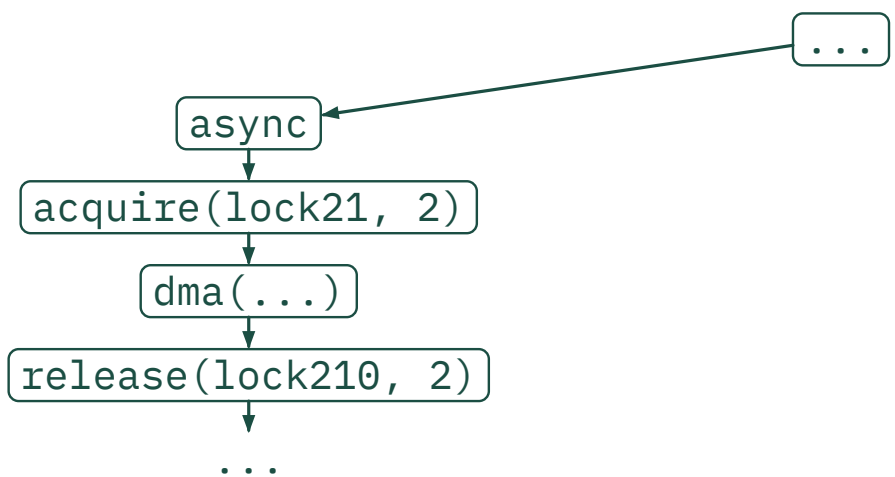


Bug Example: AIE

```
async { l1: ...
  acquire(lock21, 2);
  dma(buf20, 0, 1024);
  release(lock210, 2);
  goto l1; ... }
```

```
async { l2: ...
  acquire(lock210, 1);
  dma(buf20, 0, 512, ...);
  release(lock21, 1);
  goto l2; ... }
```

```
async { l3: ...
  acquire(lock210, 1);
  dma(buf20, 8, 512, ...);
  release(lock21, 1);
  goto l3; ... }
```

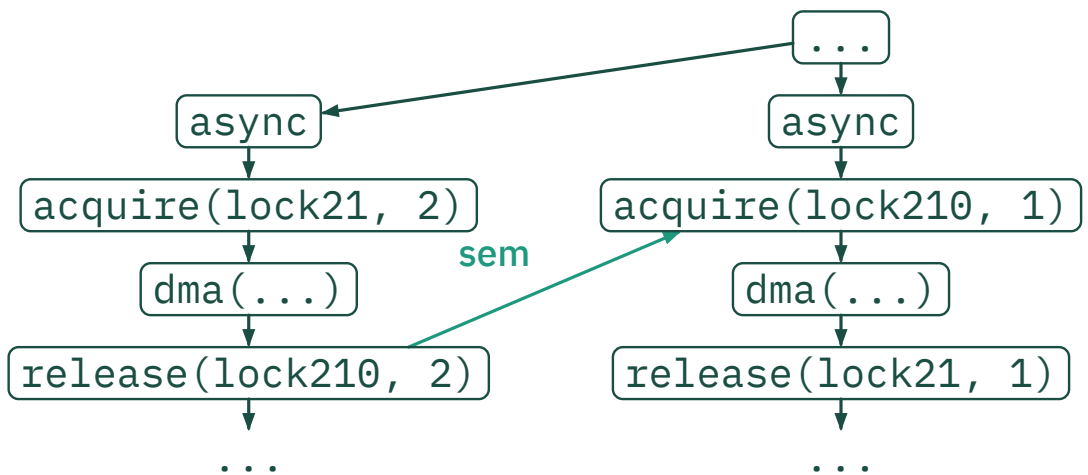


Bug Example: AIE

```
async { l1: ...
  acquire(lock21, 2);
  dma(buf20, 0, 1024);
  release(lock210, 2);
  goto l1; ... }
```

```
async { l2: ...
  acquire(lock210, 1);
  dma(buf20, 0, 512, ...);
  release(lock21, 1);
  goto l2; ... }
```

```
async { l3: ...
  acquire(lock210, 1);
  dma(buf20, 8, 512, ...);
  release(lock21, 1);
  goto l3; ... }
```

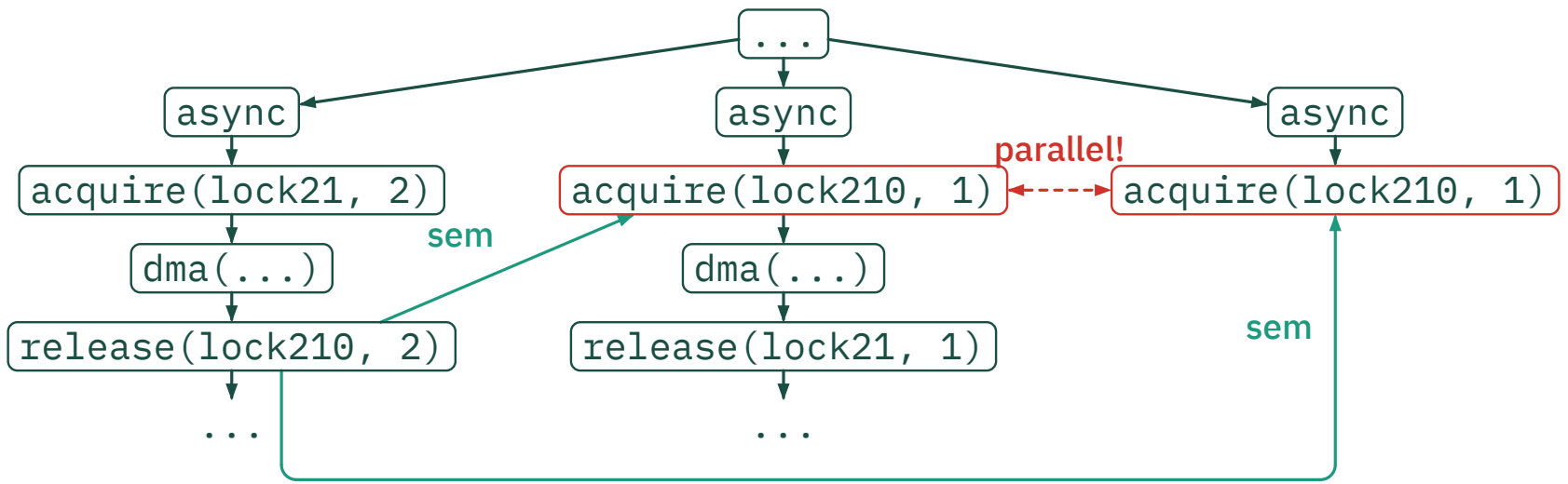


Bug Example: AIE

```
async { l1: ...
  acquire(lock21, 2);
  dma(buf20, 0, 1024);
  release(lock210, 2);
  goto l1; ... }
```

```
async { l2: ...
  acquire(lock210, 1);
  dma(buf20, 0, 512, ...);
  release(lock21, 1);
  goto l2; ... }
```

```
async { l3: ...
  acquire(lock210, 1);
  dma(buf20, 8, 512, ...);
  release(lock21, 1);
  goto l3; ... }
```



Bug reported: two parallel acquires, catches nondeterminism in this case

Experimental Results: MLIR-AIR

All 32×32 `air` files proven equivalent to their input `linalg` files

64×64: 720 tile + herd size configurations verified in ~20 min

bench	LoC	NB-S	time
`linalg`	8	201k	1.61s
`input`	17	541k	3.39s
`tiled`	102	743k	5.56s
`placed`	190	1.32M	9.01s

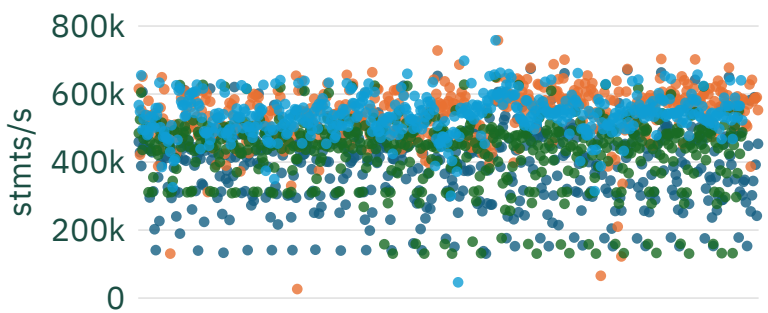
Experimental Results: MLIR Coverage

Coverage: `mlir-opt` + PolyBench, AIR `programming_examples`, …

- Parallel programs (AIR/AIE, `async`), hierarchical buffers, channels, locks, …
- Transformations: from compilers, humans, prototypes
- Often alter the memory representation and control/dataflow (**3k+ files tested**)

Throughput: about 0.5M statements/s

- PolyBench-small: < 1 s per benchmark
- Single-core equivalence check, experiments parallelize across cores: suite runs in 2 minutes



Experimental Results: Bug Finding

Polygeist: MLIR-based SCoP extraction from C to the `affine` dialect

- Is the raised MLIR program equivalent to the original C?
- Two bugs found for PolyBench: `adi` and `durbin`

`mlir-opt`: from `canonicalize` to `affine-loop-fusion`

- No comprehensive legality check for transformations: only syntactic!
- Incorrect loop fusion: `2mm` (2nd loop nest appended at the innermost loop)
- Incorrect tiling / unroll-and-jam: `floyd-warshall`
- Incorrect `polymer-opt`/PolyLib tiling for stencils (off-by-one access)

Conclusion

PEQC: correctness verification for symbolic data with concrete control flow

- **Specialized** concrete/symbolic interpretation that scales
- Concurrency model: async tasks + semaphores

PEQC-MLIR: correctness verification for MLIR programs

- Dialect semantics defined via translation to standard dialects
- Check that a research tool, prototype, or human, which may not always be correct, produced correct optimized code for a specific case
- Implemented and deployed MLIR correctness checking for AIR and AIE dialects

PEQC trades coverage for speed: *concretely interpretable* control flow

- Fit for dense linear algebra/LLM inference, (non-)affine programs with fixed loop bounds, spatial/dataflow accelerators, etc.